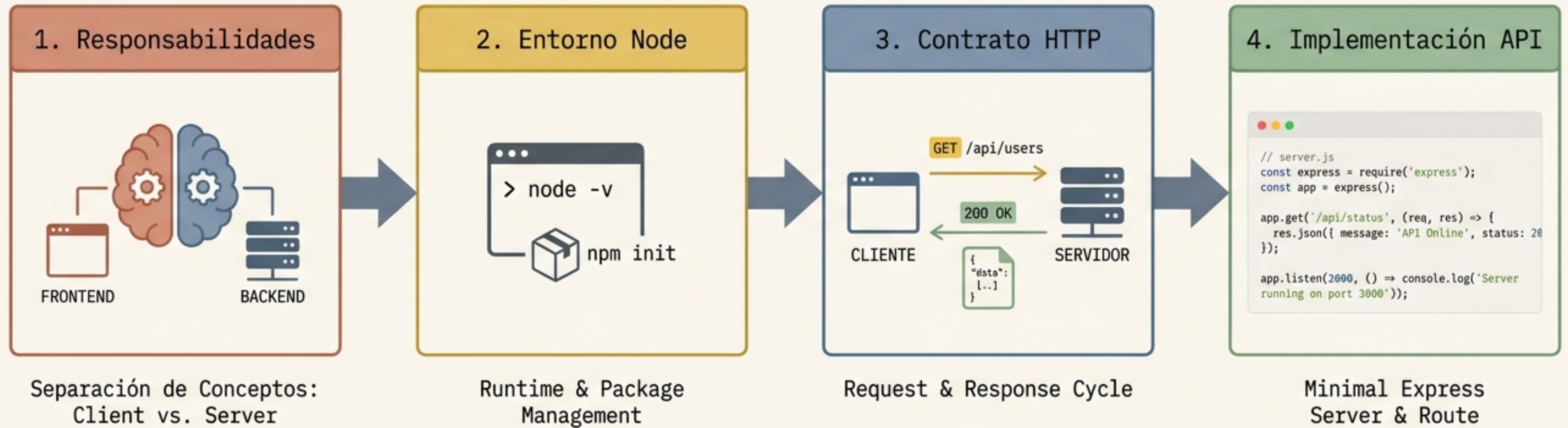


Programación Web II: Fundamentos del Backend

Objetivo Central: Entender cómo se construye y prueba una API mínima.

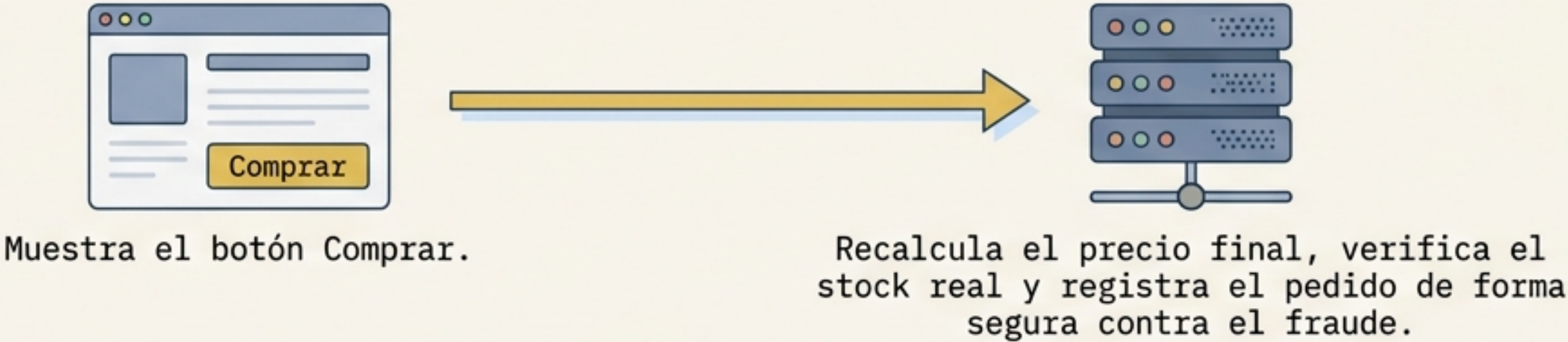


“Hoy no vamos a construir una tienda online completa; vamos a entender la base que permite que una tienda online funcione: quién muestra, quién valida, quién recibe peticiones y quién responde.”

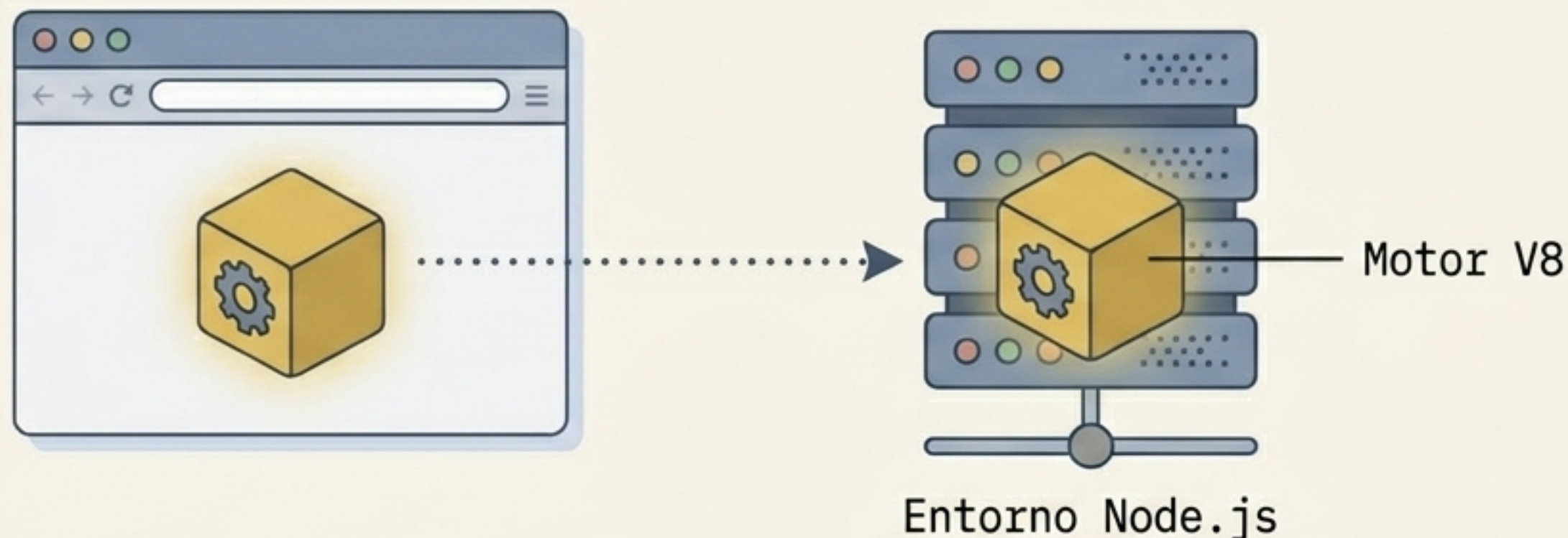
Separación de Responsabilidades: Más Allá del Navegador

Frontend (Interfaz y Experiencia)	Backend (Reglas, Datos y Seguridad)
✓ - Vive en el navegador. ✓ - Optimiza UX y feedback inmediato. ✓ - Trabaja con eventos de usuario (clicks, estados visuales).	🔒 - Vive en el servidor. 🔒 - Optimiza integridad, seguridad y consistencia. 🔒 - Trabaja con peticiones, lógica y persistencia.

Caso Real: E-commerce



El Motor: Llevando JavaScript al Servidor



¿Qué es Node?

Un runtime (entorno de ejecución) de JavaScript fuera del navegador. Permite crear **servidores** y herramientas.

Nota del Docente: Node NO es un framework de rutas (eso es Express). Es la base sobre la que se apoya todo. El navegador no puede levantar un servidor HTTP de escucha; Node sí.

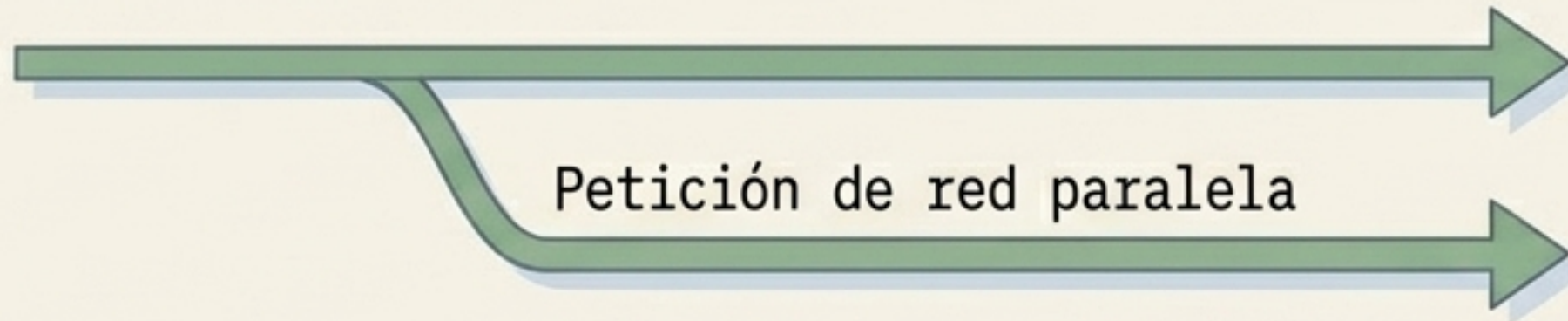
Checkpoint: El mismo lenguaje, pero un entorno radicalmente distinto con nuevas capacidades.

La Física de la Red: Asincronía para No Bloquear

Enfoque Síncrono Bloqueado



Enfoque Asíncrono

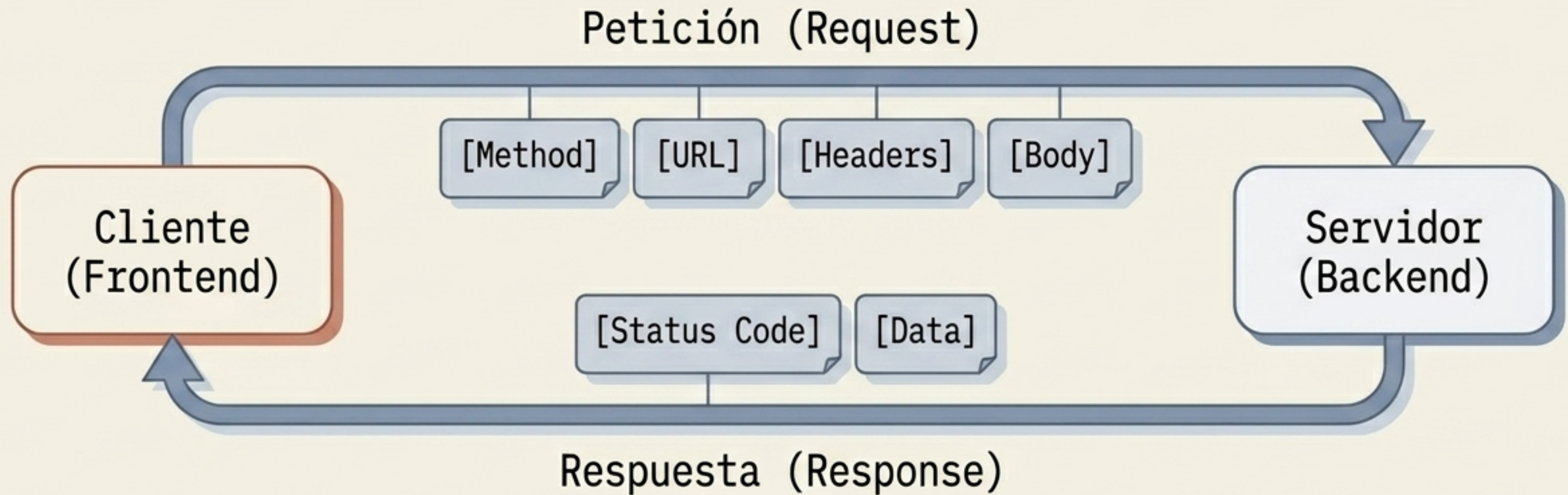


El Problema: La red, los discos y las bases de datos tardan. El programa principal no puede permitirse quedar bloqueado.

```
1 console.log('Inicio');
2
3 setTimeout(() => {
4   console.log('Respuesta simulada');
5 }, 2000);
6 console.log('Fin');
7
```

Un temporizador marca un umbral de tiempo mínimo, no una garantía de ejecución exacta. Prepara el terreno para `fetch()` y las promesas.

HTTP: El Contrato entre Sistemas Distintos



Definición: HTTP es el lenguaje común. Es un protocolo de aplicación, no solo un detalle de transporte.

Si dos equipos distintos construyen frontend y backend, este contrato técnico es el único documento que necesitan compartir para asegurar la interoperabilidad.

Matriz Diagnóstica: Métodos GET vs. POST

Método	Intención	Ubicación de Datos	Ejemplo
GET	Consultar o recuperar datos.	URL y Query Params.	/products?category=books
POST	Enviar, crear o mutar recursos.	En el Body (estructurado como JSON).	/orders (con objeto JSON)

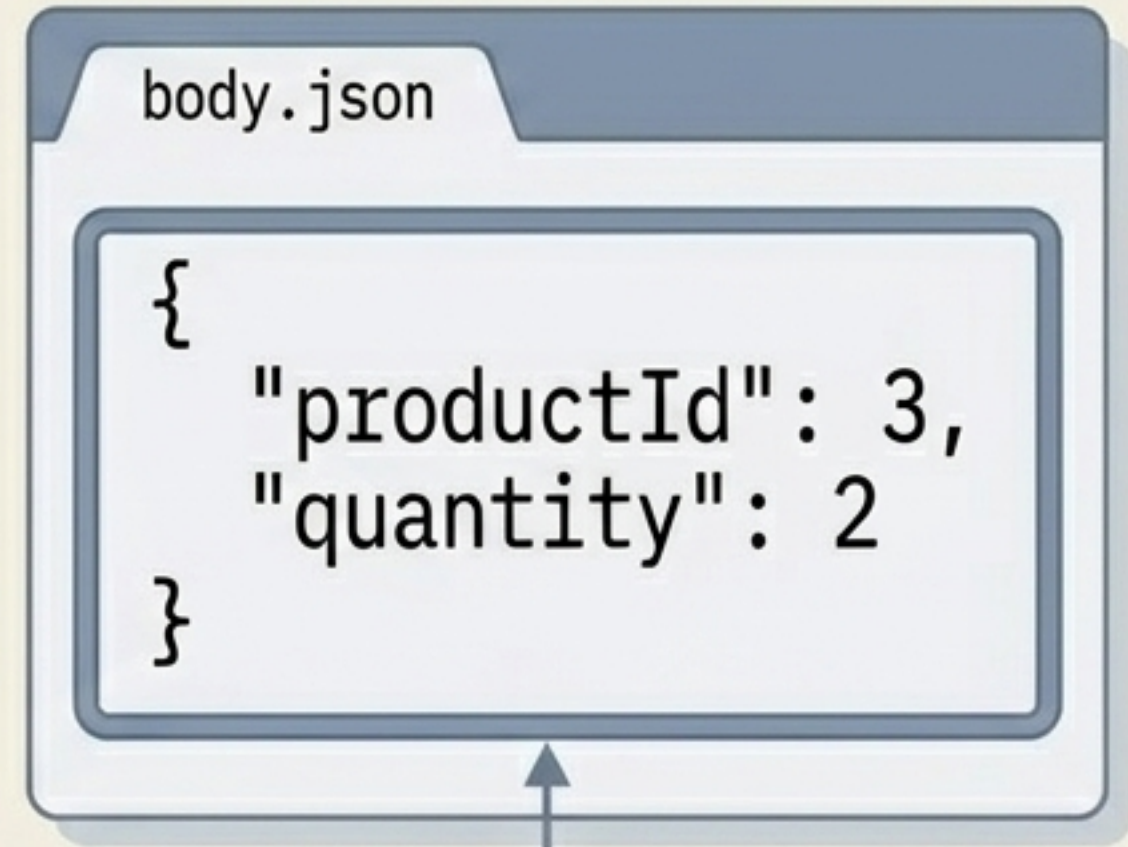


Advertencia Docente: Error muy común: Usar POST para todo simplemente "porque funciona". La elección del método comunica una intención semántica estricta al resto del equipo.

Anatomía de los Datos: Query Params vs. Body

localhost:3000/products
?category=books&page=2

Query Params (Filtros): Van al final de la URL. Ideales para filtros, orden y paginación. En Express se extraen usando req.query.



Body JSON (Carga Útil): Lleva datos estructurados. En Express se extraen usando req.body.



Regla de Oro: El Body proviene de una entrada controlada por el cliente. Debe tratarse como información NO confiable que requiere ser parseada (con express.json()) y rigurosamente validada en el backend.

Preparando el Terreno: Dependencias y Scripts

```
> npm init -y  
> npm install express
```

Comandos Iniciales:

- ``npm init -y``: Crea el manifiesto del proyecto.
- ``npm install express``: Añade la dependencia al repositorio.

```
package.json  
{  
  "name": "clase1-api",  
  "dependencies": {  
    "express": "^5.1.0"  
  },  
  "scripts": {  
    "start": "node app.js"  
  }  
}
```

❓ ¿Por qué usamos scripts como `'npm run start'` en lugar de ejecutar comandos a mano? Porque asegura consistencia y automatiza procesos repetibles para todo el equipo.

Implementación: El Primer Servidor GET

Contexto: Express es un framework web mínimo que mapea directamente métodos HTTP a 'handlers' (manejadores).

```
const express = require('express');  
const app = express();
```

```
app.get('/products', (req, res) => {  
  const { category } = req.query;  
  // Lógica de filtrado...  
  res.json(data);  
});
```

Express enruta según el método HTTP elegido.

La query string (?category=...) NO forma parte de la ruta base que se define aquí.

Extracción limpia de los parámetros enviados por el cliente.

Mutando Estado: POST y Validación con Express

```
app.use(express.json());

app.post('/orders', (req, res) => {
  const { productId, quantity } = req.body;

  if (!productId || !quantity || quantity <= 0) {
    return res.status(400).json({ message: 'Datos inválidos' });
  }

  return res.status(201).json({ message: 'Pedido creado' });
});
```

Middleware: Requisito indispensable para que Express sepa leer y poblar req.body con el JSON entrante.

Validación defensiva: Si el cliente manda quantity: -100, aquí se rechaza con un código de error HTTP (400) protegiendo el sistema.

Comprobación de Contrato: Pruebas Sin Frontend

Metodología Docente: Primero probamos la API sin frontend. Esto separa el problema de la interfaz gráfica del problema del contrato HTTP. Las tres herramientas abajo expresan exactamente el mismo contrato por vías distintas.

1. Postman (Interfaz)



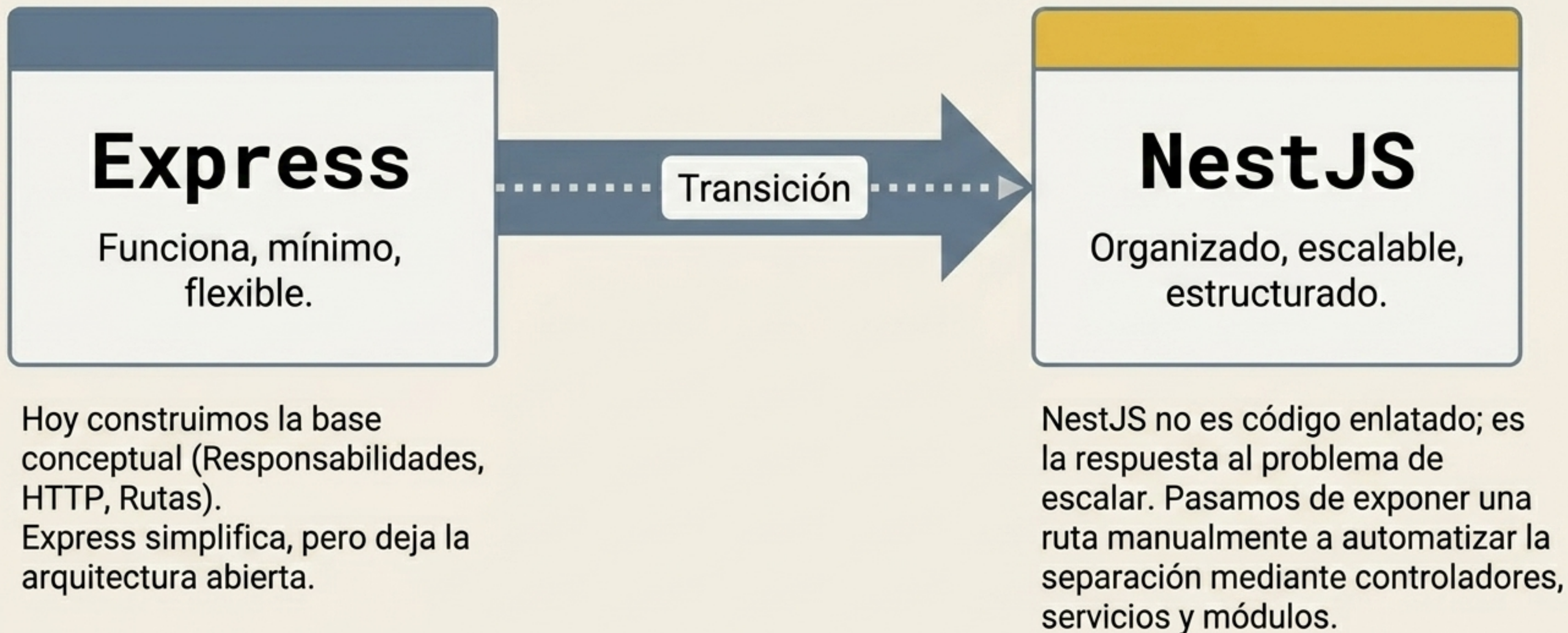
2. cURL (Terminal)

```
curl -X POST
"http://localhost:3000/orders" \
  -H "Content-Type:
application/json" \
  -d '{"productId":3,
"quantity":2}'
```

3. fetch (Navegador)

```
fetch("http://localhost:
3000/products?category=
books")\n
  .then(res => res.
json())\n
  .then(data =>
console.log(data));
```


Síntesis y Próximos Pasos: Hacia la Escalabilidad



Comprobación de Conocimiento

1. ¿Cuál de estas responsabilidades corresponde mejor al backend?

- A. Cambiar color
- B. Recalcular precio
- C. Mostrar spinner
- D. Ordenar tabla

2. ¿Qué es Node?

- A. Navegador
- B. Framework de rutas
- C. Runtime de JS fuera del navegador
- D. Gestor de paquetes

3. Si quiero filtrar productos en una API sencilla, lo más natural es usar:

- A. Body en GET
- B. Query parameter
- C. Cookie
- D. Archivo local

4. En una petición POST, los datos estructurados suelen viajar en:

- A. Path
- B. Puerto
- C. Body
- D. Hash

5. ¿Para qué sirve package.json en un proyecto Node?

- A. Guardar capturas
- B. Definir dependencias y scripts
- C. Sustituir node_modules
- D. Ejecutar navegador

Respuestas y Razonamiento

1. ¿Cuál de estas responsabilidades corresponde mejor al backend?

- A. Cambiar color
- B. Recalcular precio ✓
- C. Mostrar el menú
- D. Ordenar productos

Por qué: A, C y D son puramente visuales y de UX (frontend). El precio afecta a la integridad transaccional.

2. ¿Qué es Node?

- A. Navegador
- B. Framework de rutas
- C. Runtime de JS fuera del navegador ✓
- D. Gestor de paquetes

Por qué: No es un gestor de paquetes (eso es npm) ni un framework de rutas (eso es Express).

3. Si quiero filtrar productos en una API sencilla, lo más natural es usar:

- A. Body en GET
- B. Query parameter ✓
- C. Cookies
- D. Headers

Por qué: GET no debe llevar body. El parámetro en la URL permite leer solo el subconjunto de datos necesario.

4. En una petición POST, los datos estructurados suelen viajar en:

- A. Path
- B. Puerto
- C. Body ✓
- D. Hash

Por qué: Es el contenedor diseñado para cargas útiles seguras y estructuradas según el Content-Type.

5. ¿Para qué sirve package.json en un proyecto Node?

- A. Guardar capturas de pantalla
- B. Definir dependencias y scripts ✓
- C. Sustituir node_modules
- D. Ejecutar navegadores

Por qué: No contiene el código fuente de terceros (eso es node_modules), sino el manifiesto del proyecto.

Ejercicio Práctico: Tu Primera API

Requisitos del Proyecto

- Crear API con Express.
- GET /products: Devuelve lista JSON. Aplica filtro si llega 'category' por query.
- POST /orders: Recibe body con 'productId' y 'quantity'. Responde 400 si falta algo o quantity <= 0. Responde 201 si es exitoso.

Criterios de Evaluación

1. Arranque y escucha correcta (2 pts)
2. Uso de req.query sin ensuciar la ruta base (3 pts)
3. Lectura de req.body usando express.json() (3 pts)
4. Manejo correcto de códigos HTTP 400/201 (2 pts)

Solución de Referencia Compacta

```
app.get('/products', (req, res) => {  
  const { category } = req.query;  
  const data = category ? products.filter(p =>  
    p.category === category) : products;  
  res.json(data);  
});  
  
app.post('/orders', (req, res) => {  
  const { productId, quantity } = req.body;  
  if (!productId || !quantity || quantity <= 0) {  
    return res.status(400).json({ message: 'Datos  
inválidos' });  
  }  
  return res.status(201).json({ message: 'Pedido  
creado' });  
});
```

Nota Docente: La solución subraya los 4 pilares: separación de query string, extracción de variables, parseo JSON y validación.